# Static Timing Analysis

Part 4 – Timing Constraints

Amr Adel Mohammady

/amradelm

# Save The Palestinian Children

Till Nov,2023

Israel has killed more than 13,000 children in Gaza since October 7 while others are suffering from severe malnutrition and do not “even have the energy to cry”, says the United Nations Children’s Fund (UNICEF).

“Thousands more have been injured or we can’t even determine where they are. They may be stuck under rubble … **We haven’t seen that rate of death among children in almost any other conflict in the world**”

-**UNICEF** Executive Director

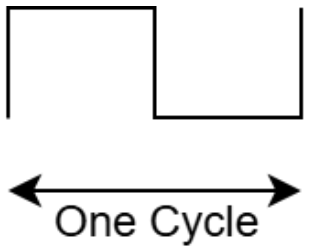
# Introduction

- In part 1 we went through the basic principles that are needed to understand all VLSI timing checks.
- In part 2 we looked into setup and hold checks and how to fix the violations.
- In part 3 we discussed other timing checks such as max transition, max capacitance, skew, etc.
- In this part we will learn how to apply timing constraints on the design.

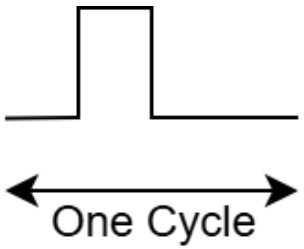
# Clock Constraints

# Clock Constraints

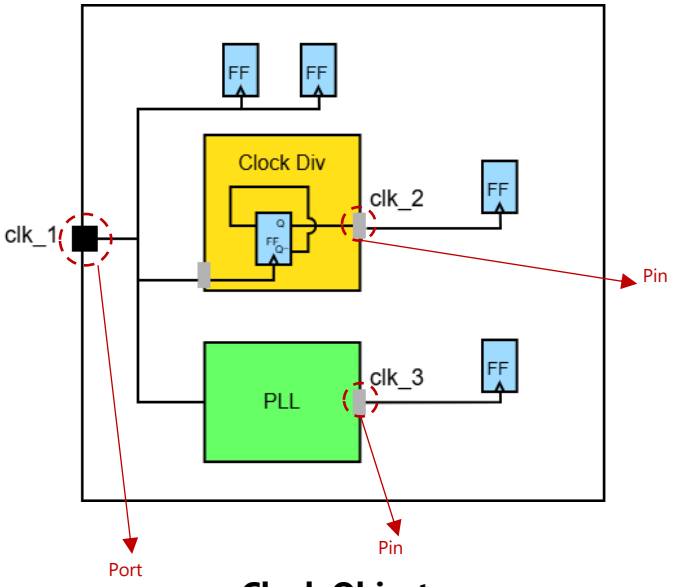
- The most important timing constraint is the clock and its period.
- **The command to define a clock is** `create_clock` **We have to define multiple things for each clock in the design**:
  - Period :The only required argument. Others are optional.
    - `create_clock -period <arg>`
  - Waveform : The rise and fall edges over one cycle.
    - `create_clock -period 4 -waveform {1 2}`
      `#The first time specified (arg1) represents the time of the first rising transition, and the second time specified (arg2) is the falling edge`
  - Object : The source the clock comes from. It can be a port or a pin (for example, a PLL output)
    - `create_clock -period <arg> [get_port my_clock_port]`




Period 4 : Waveform {0 2}
#Default



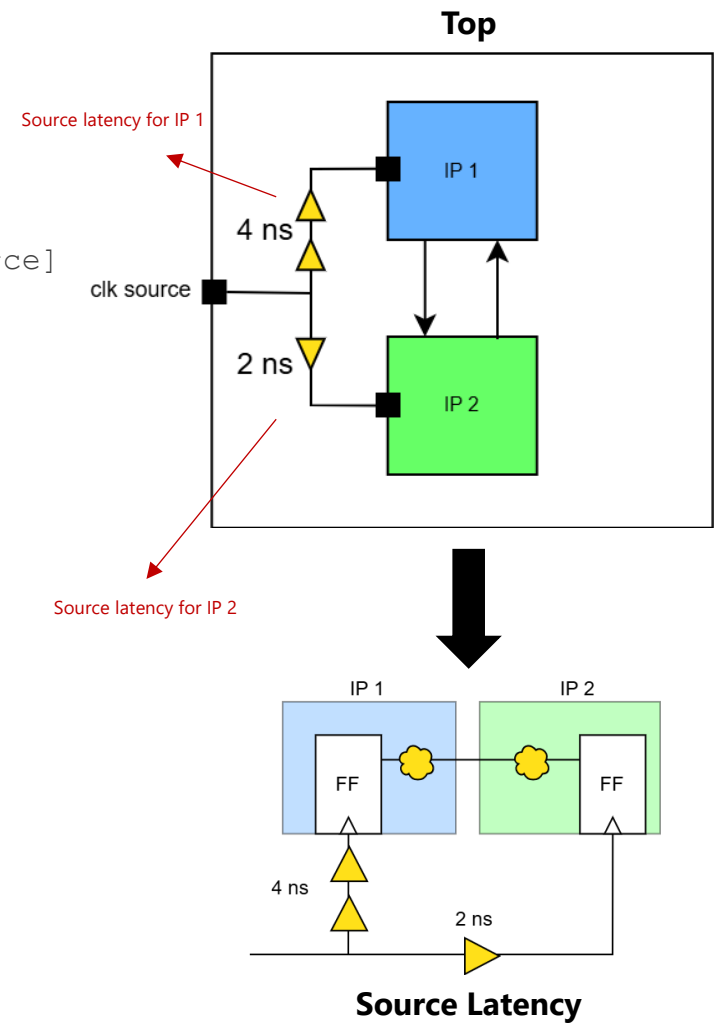

Period 4 : Waveform {1 2}




**Clock Objects**

# Source Latency

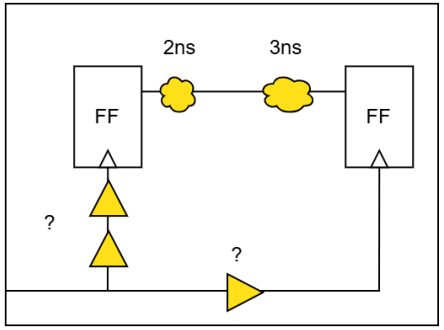
- Source latency :
  - The latency/delay from the clock outer source till it reaches the IP/module clock port.
  - This becomes very important when doing hierarchical flow[1] as it defines the local skew between the modules.
  - `set_clock_latency [-clock <args>] [-rise] [-fall] [-late] [-early] [-source] <latency> <objects>`
    - `[-clock <args>]`: The clock that will get the latency
    - `[-rise] [-fall]`: The latency during rising edge or falling edge
    - `[-late] [-early]`: The latency is applied to early or late analysis?
    - `[-source]`: Is it a source or network latency? Network latencies will be explained soon.
    - `<objects>`: The clock source object (pin or port)



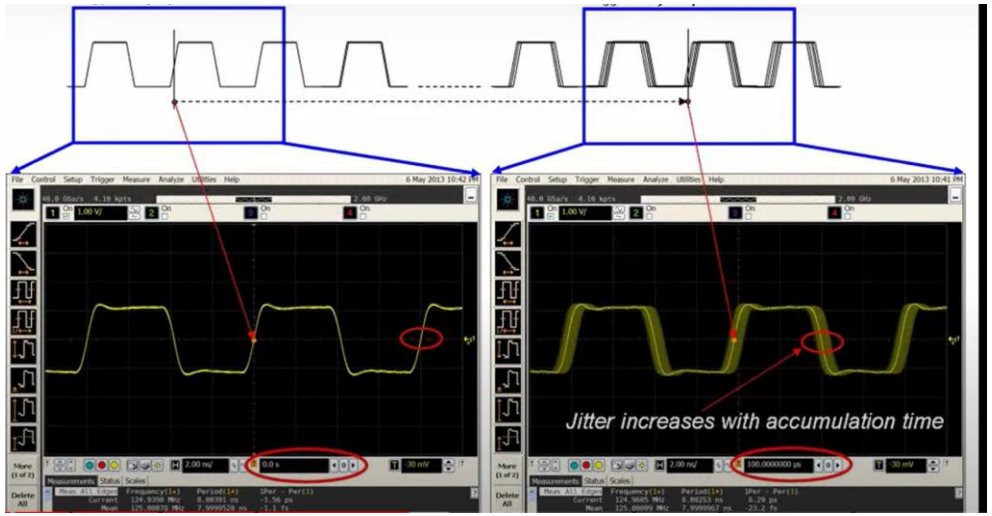

[1]: Hierarchical flow refers to when the modules of the design are synthesized and hardened separately and then integrated later in a top module.

# Clock Uncertainty

- Clock Uncertainty :
  - From its name, clock uncertainty defines a worst-case values about things we are not certain about regarding the clock.
  - For example, What is the clock skew between 2 flip-flops during synthesis? We can't know because there is no clock tree yet.
  - Therefore we will assume a clock tree skew to be used during synthesis.
  - Also the Oscillator/PLL generating the clock doesn't generate a clean clock with a constant period. Instead, the generated clock has an error margin meaning that the clock period changes over time.
    - Imagine you have a drum that beats regularly, like a clock.
    - Normally, it goes "beat, beat, beat" at the same time intervals.
    - If sometimes it goes "beat... beat..... beat," faster or slower than it should, that's jitter.
    - Each time you count the time between beats (period), it's a little different.




**Skew Is Unknown In Synthesis**

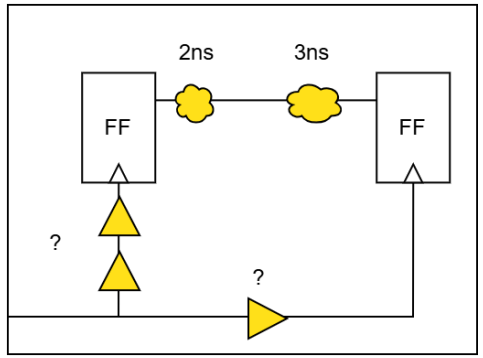



**Ideal Clock** **Clock with jitter**

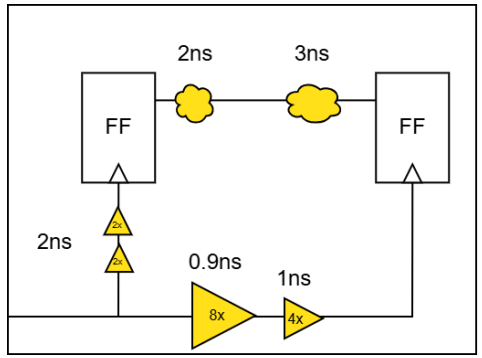
[1]: Jitter Part 3: C2C Jitter and Long Term Jitter (youtube.com)

# How to Apply Clock Uncertainty

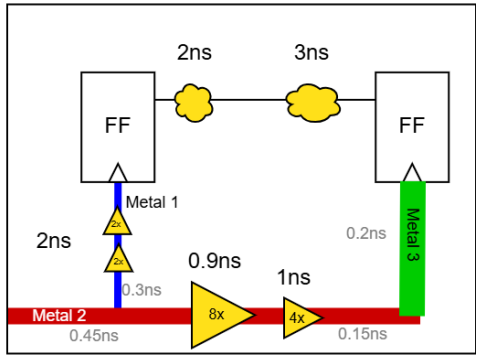
- During Synthesis we don't have a clock tree so we need to assume a clock skew. Some designers assume 20% of the clock period but this value can change depending on the project.
  - ❑ `set_clock_uncertainty [expr $pll_jitter + $clock_period*0.20] [get_clocks myClk] #Uncertainty in Synthesis`
- After CTS there is a clock tree and we know the cell delays but we don't have clock routes yet[1]. We can relax the clock uncertainty and leave a term to account for the routes.
  - ❑ `set_clock_uncertainty [expr $pll_jitter + $clock_period*0.10] [get_clocks myClk] #Uncertainty After CTS`
- After route stage we know everything about the clock tree but we still have 2 sources of uncertainties: The PLL jitter and the network jitter[2].
  - ❑ `set_clock_uncertainty [expr $pll_jitter + $clock_period*0.03] [get_clocks myClk] #Uncertainty After Route`



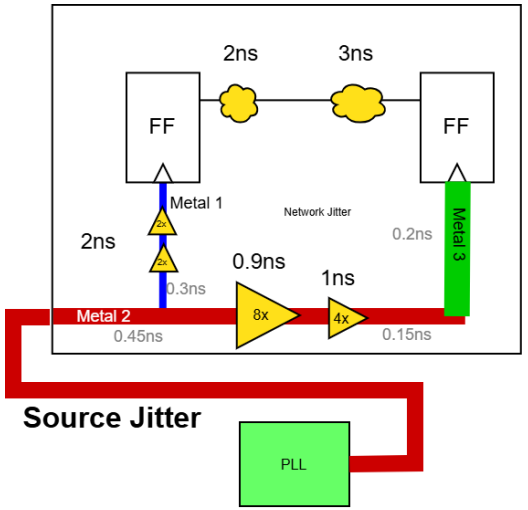

**During Synthesis**




**After CTS**




**After Route**

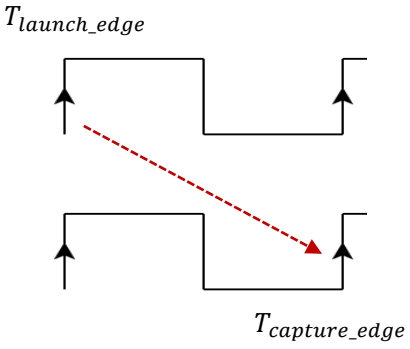



[1]: We can instruct the tool to route the clock nets in CTS stage. We still have uncertainty in the routes because we don't know the crosstalk and coupling capacitance that will happen after we route the other signal nets. So we need to account for this uncertainty.

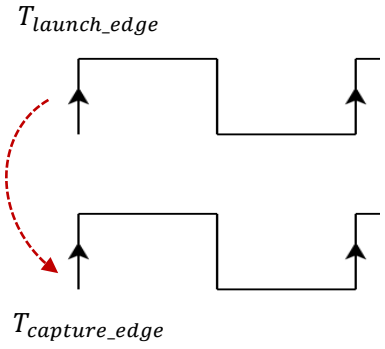
[2]: The network jitter is the jitter that happens due to the varying delays of the cells and routes due to temperature and voltage variations over time. This value is very small and can range from 1%-5% of the clock period for advanced tech nodes.

# Clock Uncertainty

- The pll jitter happens from one cycle (one edge) to another. Therefore the pll jitter is only applied for setup paths where the launch and capture edges are different but not applied on full cycle hold paths because the edges are the same.
- We still apply some uncertainty on full cycle hold as a safety margin.
  - ❑ `set_clock_uncertainty -setup [expr $pll_jitter + $clock_period*0.03] [get_clocks myClk] #Setup`
  - ❑ `set_clock_uncertainty -hold [expr $clock_period*0.03] [get_clocks myClk] #Hold`



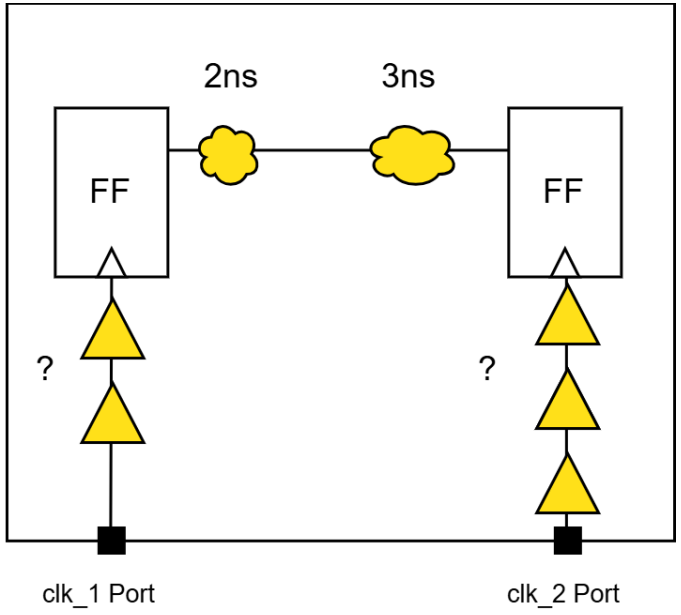

**Setup Edges for Full Cycle Paths**




**Hold Edges for Full Cycle Paths**

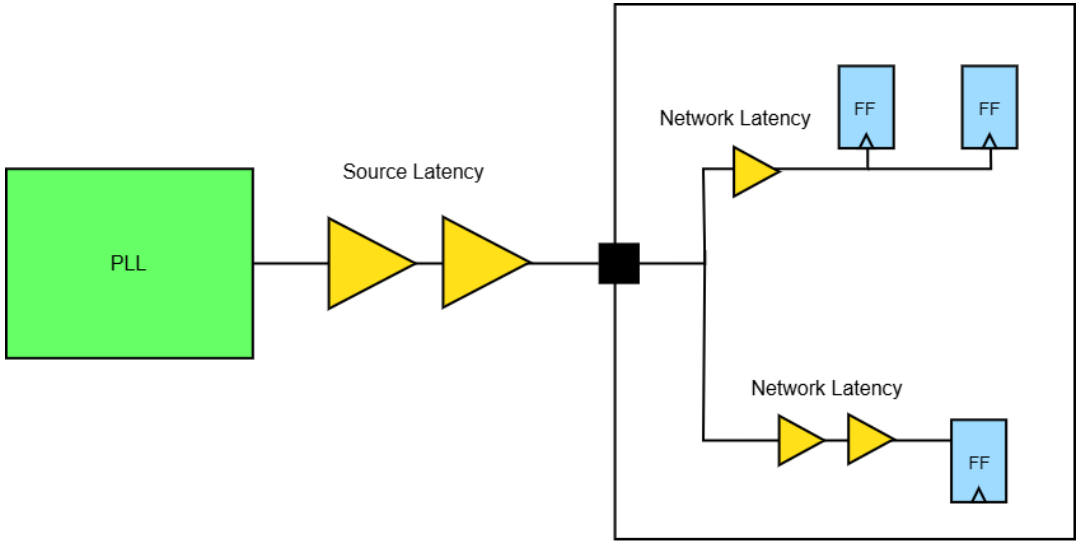
# Clock-To-Clock Uncertainty

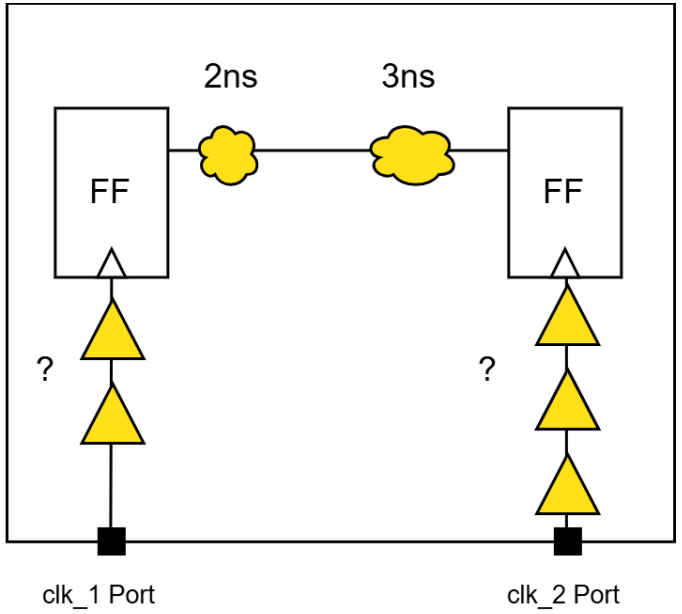
- The uncertainty between two different clocks can be defined using the `set_clock_uncertainty` command
  - `set_clock_uncertainty 2.25 -from [get_clocks clk_1] -to [get_clocks clk_2]`
  - `set_clock_uncertainty 1.50 -from [get_clocks clk_2] -to [get_clocks clk_1]`



# Network Latency

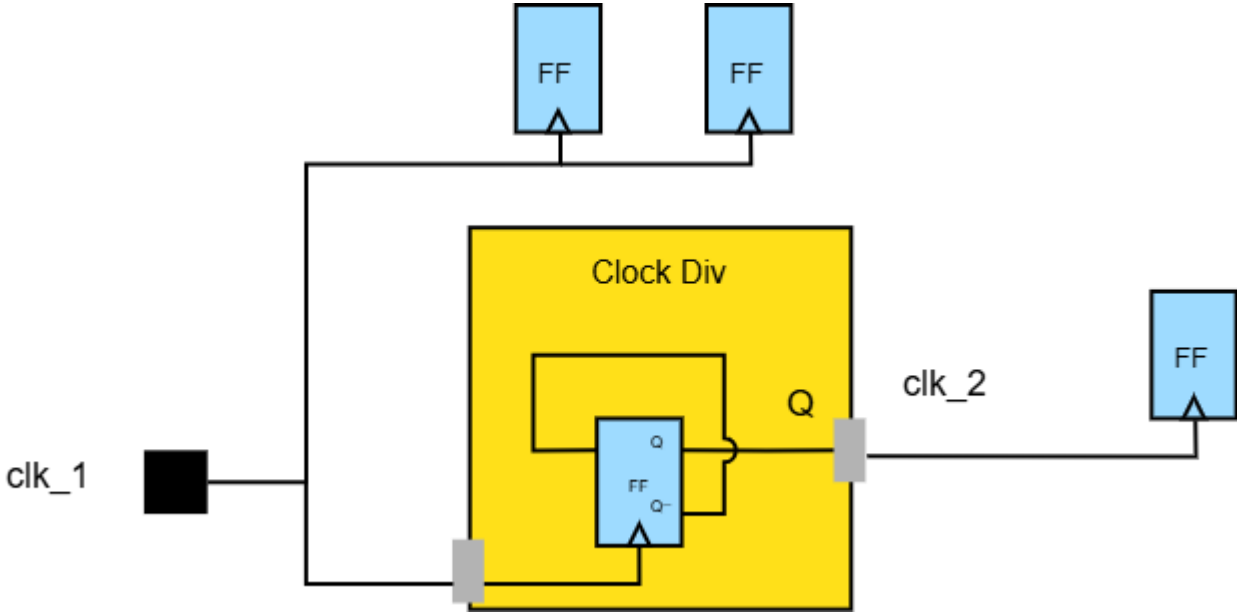
- The network latency defines the clock delay within the block while the source latency defines the delay outside the block.
- Before CTS, network latency can model the expected latency of the clock tree.
- This helps the tool optimize the timing of the design especially for the IO paths and the paths going from and to different clocks.
- In the diagram on the right, we can apply a network latency of 2 ns to clk_1 and 5ns to clk_2 which is equivalent of applying a skew of $|2 - 5| = 3$
  - ~~`set_clock_latency [-source] 2.00 [get_port clk_1]`~~
  - ~~`set_clock_latency [-source] 5.00 [get_port clk_2]`~~



# Generated Clock

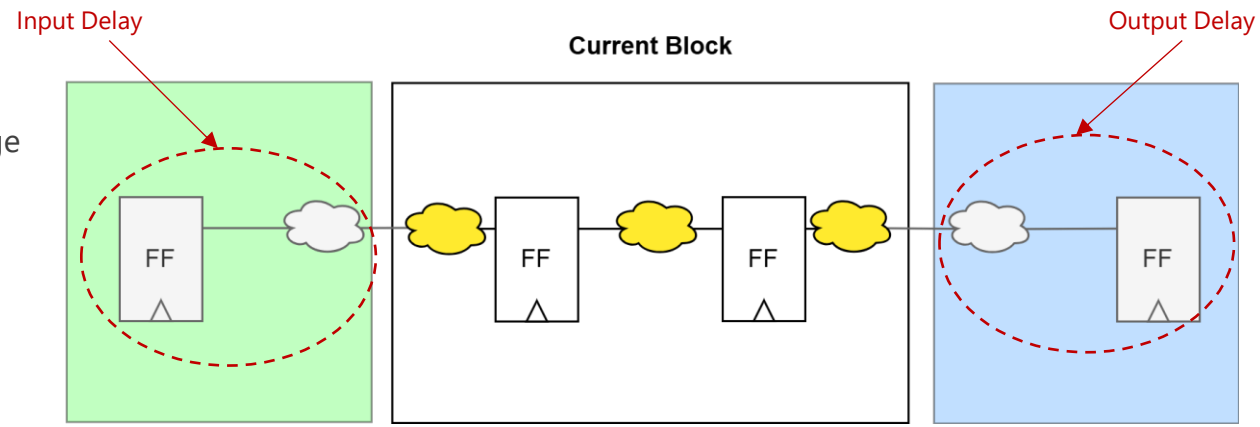
- If a primary clock goes through a clock generator such as a clock divider then the divided clock is said to be a “generated clock”.
- The generated clocks can be identified automatically by the STA tools or defined manually by the designer.
- It’s better to define it manually to avoid any discrepancy when using different tools.
- **To define the generated clock manually:**
  - `create_generated_clock -source [get_ports clk_1] -divide_by 2 -add -name CLK2 [get_pins clock_div/Q]`



# I/O Budgeting

# I/O Budgeting

- The current block communicates with other blocks through the I/O ports.
- To fully run STA on the block we need to inform the tool about the delays in the other blocks so that when we integrate the blocks we don't get surprises and we meet timing.
- This is called I/O budgeting or I/O delays.
- **The delay coming from input ports is called input delay and it consists of:**
  - $T_{cq}$ of the launching FF
  - Combinational path delay in the launching block.
  - Net delay between the two blocks.
- **The delay coming from output ports is called output delay and it consists of:**
  - $T_{setup}$ of the capturing FF
  - Combinational path delay in the capturing block.
  - Net delay between the two blocks.
- Most designers set 50% of clock period as an IO delay. But this can change from one project to another



# I/O Budgeting – Effect of Clock Latency

- **The only terms missing are the clock latencies to the launching and capturing flip-flops.**
- **There are different ways to handle the latency when dealing with IOs:**
  - Apply zero latencies and include the latency in the IO delay value (As in the reports below)[1]
  - Specify the latency on the ports manually.
  - Apply the median clock latency

```
Startpoint: exrw (input port clocked by EXCLK)
Endpoint: pin_exrw (rising edge-triggered flip-flop clocked by
PLL_33M:pll_ex|altpll:altpll_component|_clk0)
Path Group: PLL_33M:pll_ex|altpll:altpll_component|_clk0
Path Type: max

Point                                                    Trans      Incr       Path
-------------------------------------------------------------------------------------
clock EXCLK (rise edge)                                              0.000      0.000
clock network delay (propagated)                                     0.000      0.000
input external delay                                                15.000     15.000 f
exrw (in)                                                 2.640      0.000 &   15.000 f
pin_exrw/PIN (C67002_0000000F90C1040298200000108_V33_LVTTL)   2.640  0.027 H   15.027 f
pin_exrw/PINin (C67002_0000000F90C1040298200000108_V33_LVTTL) 2.640    0.000    15.027 f
pin_exrw/DATOVR (C67002_0000000F90C1040298200000108_V33_LVTTL) 0.255   0.772 H   15.799 f
data arrival time                                                                15.799

clock PLL_33M:pll_ex|altpll:altpll_component|_clk0 (rise edge)      30.303     30.303
clock network delay (propagated)                                     0.331     30.634
clock reconvergence pessimism                                        0.000     30.634
inter-clock uncertainty                                             -0.260     30.374
pin_exrw/CLKIN (C67002_0000000F90C1040298200000108_V33_LVTTL)                   30.374 r
library setup time                                                  -3.978     26.396
data required time                                                             26.396
-------------------------------------------------------------------------------------
data required time                                                             26.396
data arrival time                                                             -15.799
-------------------------------------------------------------------------------------
slack (MET)                                                                    10.597
```

```
Startpoint: pin_pn_cnt1
              (rising edge-triggered flip-flop clocked by pll_tx_mode)
Endpoint: pn_cnt1_b (output port clocked by in_lvds_mode)
Path Group: in_lvds_mode
Path Type: max

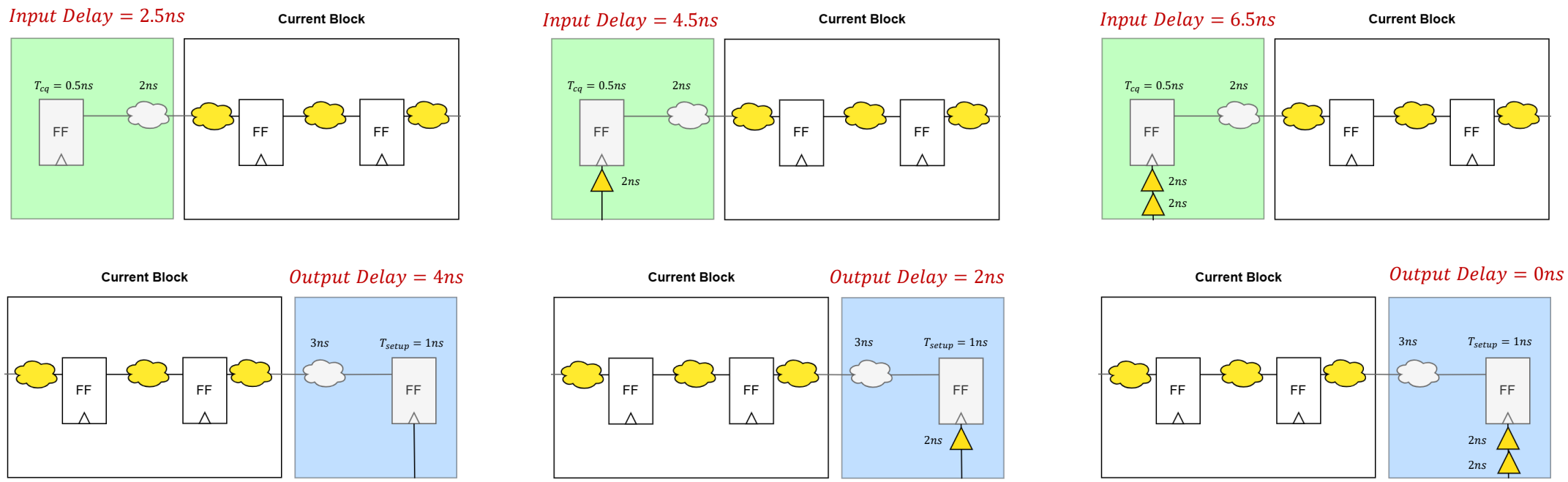
Point                                                Trans       Incr       Path
-------------------------------------------------------------------------------
clock pll_tx_mode (rise edge)                                    0.000      0.000
clock network delay (propagated)                                 0.929      0.929
pin_pn_cnt1/CLKOUT (C680043_000000000000C40298000060420_V33_LVCMOS)
                                                     0.076       0.000      0.929 r
pin_pn_cnt1/DIN (C680043_000000000000C40298000060420_V33_LVCMOS)
                                                     0.138       0.966 &    1.894 f
pin_pn_cnt1/PIN (C680043_000000000000C40298000060420_V33_LVCMOS)
                                                     0.824       1.705 &    3.599 f
pn_cnt1 (out)                                        0.824       0.000 &    3.599 f
data arrival time                                                           3.599

clock in_lvds_mode (rise edge)                                  12.642     12.642
clock network delay (propagated)                                 0.000     12.642
clock reconvergence pessimism                                    0.000     12.642
inter-clock uncertainty                                         -0.240     12.402
output external delay                                           -8.090      4.312
data required time                                                          4.312
-------------------------------------------------------------------------------
data required time                                                          4.312
data arrival time                                                          -3.599
-------------------------------------------------------------------------------
slack (MET)                                                                 0.713
```

[1]: Reference - https://cdrdv2-public.intel.com/655075/an554.pdf

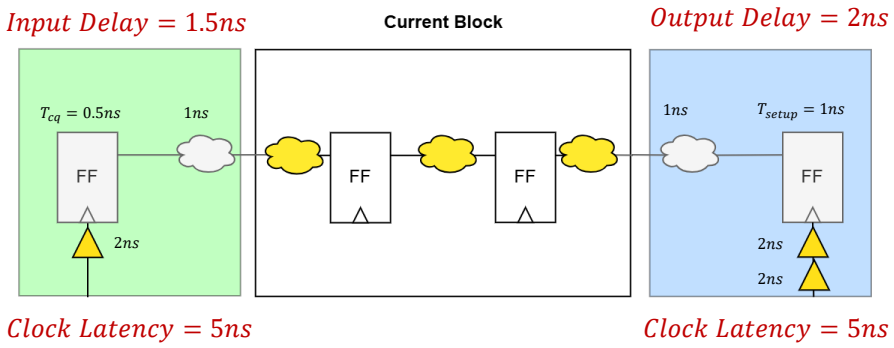
# I/O Budgeting – Effect of Clock Latency – Zero Latency

- **In the case of zero latency, you have to account for the latency inside the IO delay value:**
  - For Input delay: latency is added to the combinational delay value
  - For output delay: latency is subtracted from the combinational delay value.
- **The command to apply IO delay while including latency:**
  - `set_input_delay 6.5 -clock CLK1 -network_latency_included [all_inputs]`



# I/O Budgeting – Effect of Clock Latency – Manual Latency

- The other approach is to apply a fixed value for the clock latency.
- **To apply IO delay without including latency then applying the fixed latency:**
  - `set_input_delay 1.5 -clock CLK1 [get_port my_port]`
  - `set_output_delay 2 -clock CLK1 [get_port my_port]`
  - `set_clock_latency 5 [get_clock CLK1]`[1]
- **To apply latency on a specific port:**
  - `set_clock_latency 5 -clock [get_clock CLK1] [get_port my_port]`




Propagated latency applied on clock pins

Ideal latency applied on ports

```
Point                                                    Incr       Path
-----------------------------------------------------------------------------
clock clk (rise edge)                                    0.00       0.00
clock network delay (propagated)                         1.64       1.64
I_RISC_CORE/I_REG_FILE/REG_FILE_A_RAM/CE2 (ram16x128)
                                                         0.00       1.64 r
I_RISC_CORE/I_REG_FILE/REG_FILE_A_RAM/IO2[14] (ram16x128)
                                                         1.91       3.55 f
I_RISC_CORE/I_REG_FILE/RegPort_A[14] (REG_FILE)          0.00       3.55 f
I_RISC_CORE/RESULT_DATA[14] (RISC_CORE)                  0.00       3.55 f
U23/Z (xr02d1)                                           0.25 *     3.80 f
U9/Z (xr03d1)                                            0.34 *     4.15 r
U49/ZN (inv0d1)                                          0.05 *     4.20 f
U37/Z (mx02d2)                                           0.23 *     4.43 f
U47/ZN (invbd7)                                          0.12 *     4.55 r
RESULT_DATA_iopad_3/PAD (pc3o05)                         2.01 *     6.56 r
RESULT_DATA[3] (out)                                     0.00 *     6.56 r
data arrival time                                                   6.56

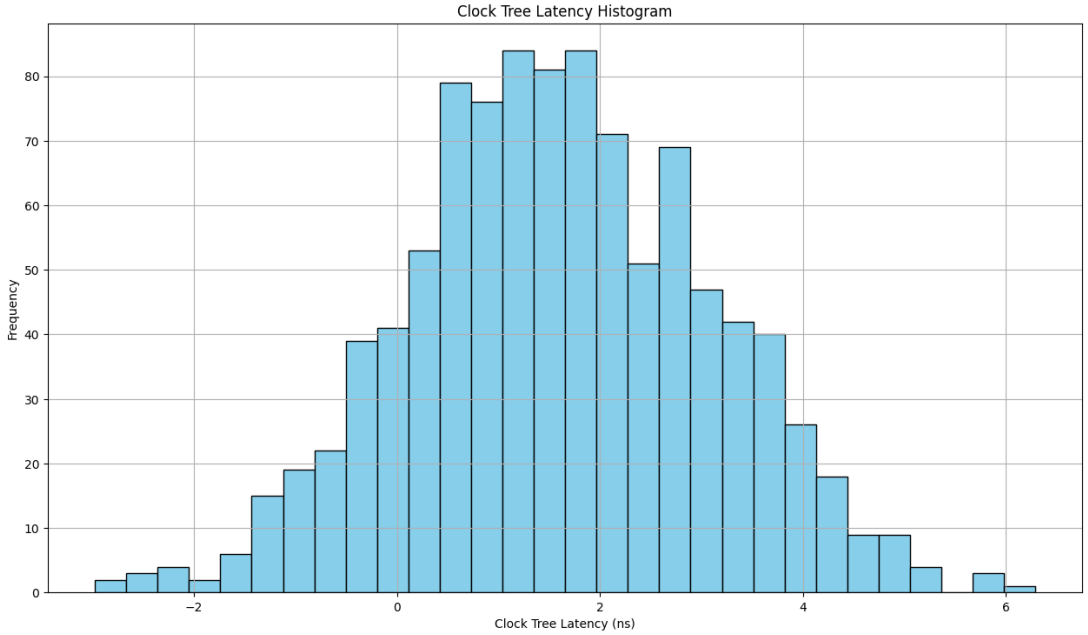
clock clk (rise edge)                                    8.00       8.00
clock network delay (ideal)                              5.00      13.00
clock uncertainty                                       -4.00       9.00
output external delay                                   -2.00       7.00
data required time                                                  7.00
-----------------------------------------------------------------------------
data required time                                                  7.00
data arrival time                                                  -6.56
-----------------------------------------------------------------------------
slack (MET)                                                         0.44
```

**SNPS Timing Report : Fixed Ideal Latency**

[1]: The tools will use this value for ports while the rest of the design will use the actual clock tree values (if the clock is set as propagated and not ideal)

# I/O Budgeting – Effect of Clock Latency – Median Latency

- The other approach is to use the median latency of all the registers in the design after building the clock tree.
- **The command to apply IO delay without including latency then apply the median:**
  - `set_input_delay 1.5 -clock CLK1 [all_inputs]`
  - `set_output_delay 2 -clock CLK1 [all_outputs]`
  - Then after CTS:
  - `compute_clock_latency`[1]




```
Point                                                   Incr       Path
-----------------------------------------------------------------------
clock clk (rise edge)                                   0.00       0.00
clock network delay (propagated)                        1.64       1.64
I_RISC_CORE/I_REG_FILE/REG_FILE_A_RAM/CE2 (ram16x128)
                                                        0.00       1.64 r
I_RISC_CORE/I_REG_FILE/REG_FILE_A_RAM/IO2[14] (ram16x128)
                                                        1.91       3.55 f
I_RISC_CORE/I_REG_FILE/RegPort_A[14] (REG_FILE)         0.00       3.55 f
I_RISC_CORE/RESULT_DATA[14] (RISC_CORE)                 0.00       3.55 f
U23/Z (xr02d1)                                          0.25 *     3.80 f
U9/Z (xr03d1)                                           0.34 *     4.15 r
U49/ZN (inv0d1)                                         0.05 *     4.20 f
U37/Z (mx02d2)                                          0.23 *     4.43 f
U47/ZN (invbd7)                                         0.12 *     4.55 r
RESULT_DATA_iopad_3/PAD (pc3o05)                        2.01 *     6.56 r
RESULT_DATA[3] (out)                                    0.00 *     6.56 r
data arrival time                                                  6.56

clock clk (rise edge)                                   8.00       8.00
clock network delay (ideal)                             1.63       9.63
clock uncertainty                                      -4.00       5.63
output external delay                                  -2.00       3.63
data required time                                                 3.63
-----------------------------------------------------------------------
data required time                                                 3.63
data arrival time                                                 -6.56
-----------------------------------------------------------------------
slack (VIOLATED)                                                  -2.93
```

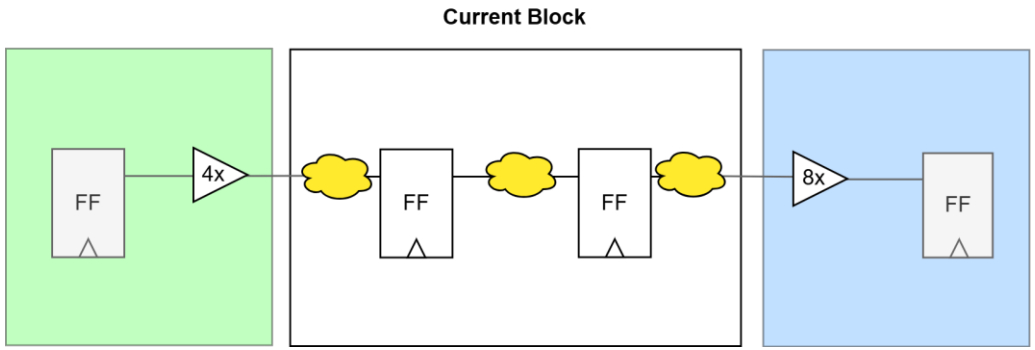
**SNPS Timing Report : After Latency Update**

[1]: The command works for SNPS ICC2 and Fusion Compiler.

# I/O Budgeting – Input Transition And Load Capacitance

- The calculation of the delay of the cells just at the input port needs the input transition time. Unless specified, the tool will assume ideal (zero) transition time which might cause timing violation when the blocks are integrated.
- **There are 2 ways to manually define the input transition:**
  - Using a fixed numeric value
    - `set_input_transition 0.75 [get_ports DATA_IN*]`
  - Using the drive strength of a cell (for example, assume the cell that drive the port is a buffer of size 4)
    - `set_driving_cell -lib_cell BUF_4 [get_ports DATA_IN*]`

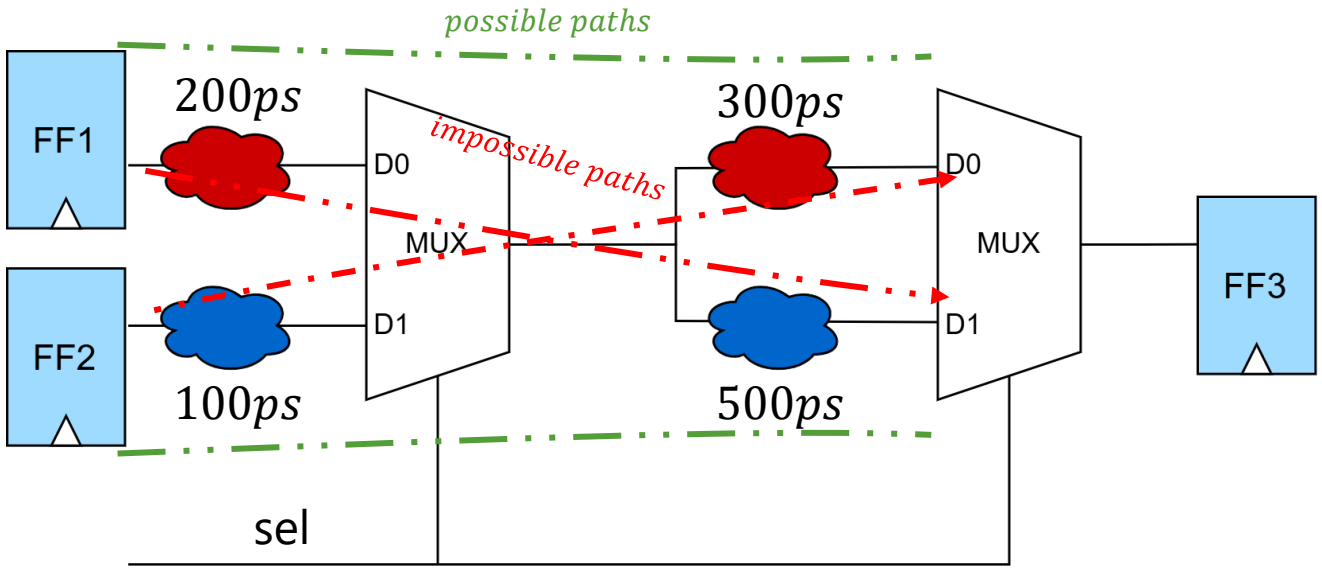
- Similarly the cell that drives the output port needs the load capacitance value to calculate its delay.
- **There are 2 ways to define the load cap:**
  - Using a fixed cap value
    - set_load 3 [all_outputs]
  - Using the load cap of a cell (for example, assume the load cell is a buffer of size 8)
    - set pin_cap [get_attribute **[get_lib_pins tech_lib/BUF_8/A]** pin_capacitance]
      set_load $pin_cap [all_outputs]



# Path Based Constraints

# False Paths

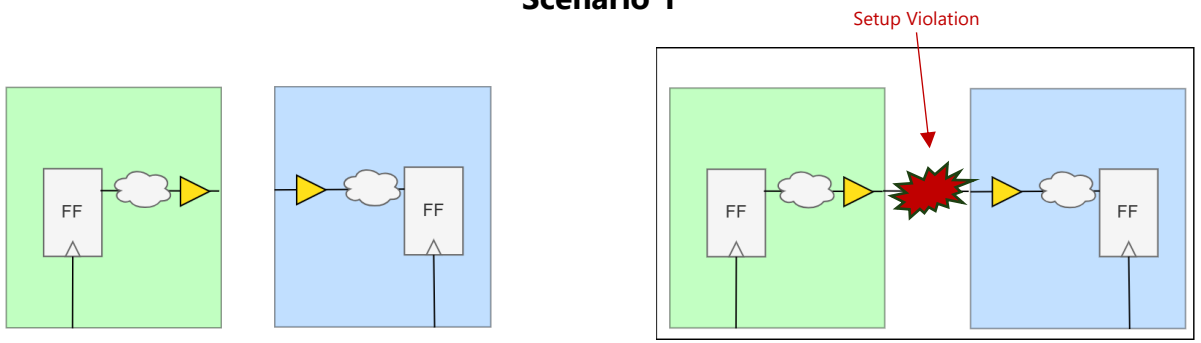
- False paths are timing paths that can't possibly occur due to the logic of the circuit
- Applying false path constraints requires a good understanding of the functional operation of the circuit.
  - ```
    set_false_path -from FF1/CP -through {MUX_1/D0} -through {MUX_2/D1} -to FF3/D
    set_false_path -from FF2/CP -through {MUX_1/D1} -through {MUX_2/D0} -to FF3/D
    ```


    ```

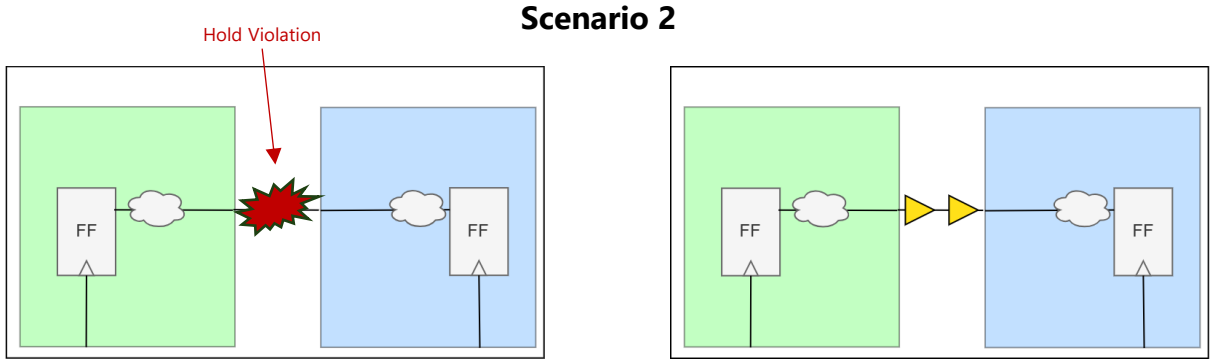
# False Paths – Hold on IO Ports

- **Some designers apply false path for the hold analysis on IO ports for sub-blocks during hierarchal flow**
  - `set_false_path -hold -from [all_inputs]`
  - `set_false_path -hold -to [all_outputs]`
- This is because hold can be easily fixed in the top module by adding buffers
- **To understand this lets consider these 2 scenarios**: Two engineers each are working on their block:
  - Scenario 1:
    1. They added buffers on IO ports to fix hold violations
    2. After integration in the top module a setup violation was found
    3. Each engineer had to reopen the block, remove the buffers, reroute the nets, fix any DRCs that appeared, then write GDS and create new design libs.
    4. The top module engineer waited till both engineers finished then re-integrated the design again.
  - Scenario 2:
    1. Hold was ignored on the IO
    2. After integration a hold violation was found.
    3. The top module engineer simply added a few buffers in the top module. No redesign was needed for the subblocks.
- As you can see ignoring hold on the IO ports saves lots of work and time.

**Scenario 1**

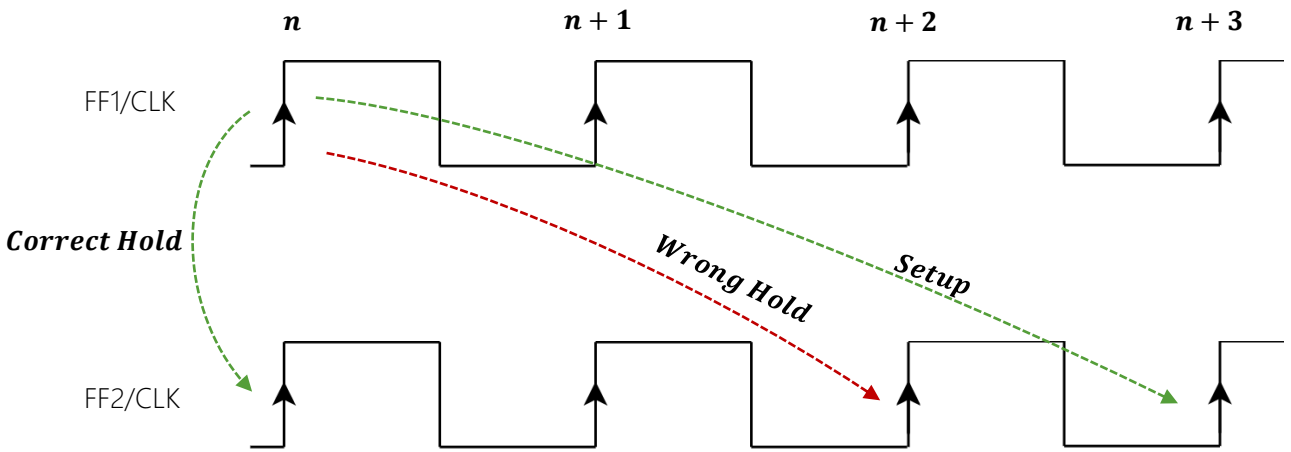



**Scenario 2**



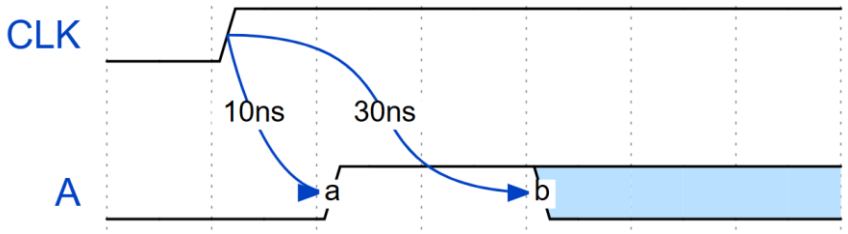
# Multi-Cycle Path

- Multi-cycle paths are timing paths that takes more than one clock cycles.
- **To set multi-cycle path:**
  - `set_multicycle_path -setup <MULTIPLIER> -from {FF1/CLK} -to {FF2/D} # MULTIPLER = 3 in the diagram below`
- STA tools will by default, select the hold capture edge to be the edge one cycle before the setup edge.
- **We fix this by instructing the tool to set hold N-1 cycles of the setup edge, where N is the number of multi-cycles.**
  - `set_multicycle_path -hold <MULTIPLIER-1> -from {FF1/CLK} -to {FF2/D} # MULTIPLER-1 = 2 in the diagram below`

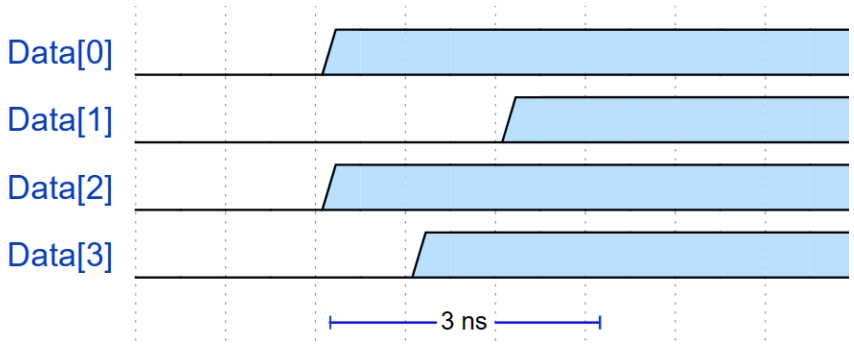


# Max/Min/Skew Delays

- As discussed in the previous part of this document, we may have max, min, or skew constraints on the design paths.
- **To apply these constraints:**
  - `set_max_delay -from FF/CLK -to Instance/A 30`
  - `set_min_delay -from FF/CLK -to Instance/A 10`
- There is no command to apply a skew constraint on the bus[1]. You need to use TCL commands to report if there is skew violation or not.
- The next slide shows an example TCL script that does skew check




**Max/Min Constraints**




**Skew Constraints**

[1]: That is in Primetime. Other tools may have commands for skew constraints

# Max/Min/Skew Delays – Skew Check Script

```
# Define skew constraint
set skew_constraint 3  ;# in ns

# Initialize min and max arrival times
set max 0
set min 999999  ;# Use a large number as an initial
"infinity" value

# Get the collection of pins
set my_pins [get_pins DATA[*]]

# Iterate over each pin in the collection
foreach_in_collection pin $my_pins {
  # Get the timing path to the pin
  set path [get_timing_path -to $pin]

  # Get the arrival time of the timing path
  set arrival [get_attribute $path arrival]
}
```

```
  # Update min and max arrival times
  if { [expr $arrival < $min] } {
    set min $arrival
  }
  if { [expr $arrival > $max] } {
    set max $arrival
  }
}

# Check if the skew constraint is violated
if { [expr ($max - $min) > $skew_constraint] } {
  puts "ERROR: Skew Violation"
} else {
  puts "INFO: Skew Passed"
```

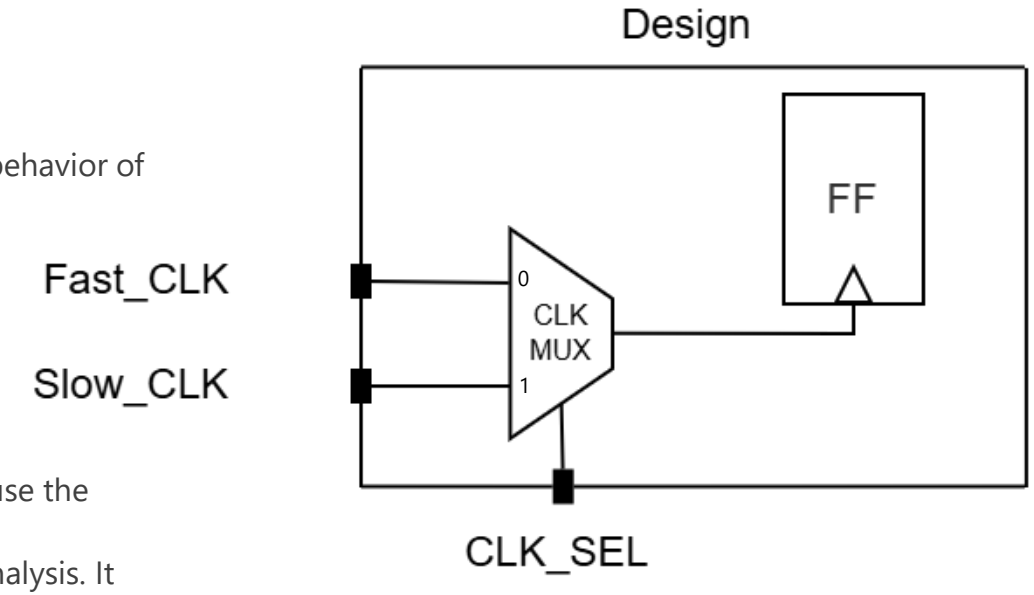
# Modes, Corners, And Scenarios

# Modes

- A mode in STA refers to a specific functional condition under which the timing analysis is performed.
- Mode Examples: Functional mode, Scan test mode, Low power mode.
- Each mode represents a distinct set of constraints and conditions that affect the timing behavior of the design.
- **For example, two modes could differ in the clock frequency.**
  - ```
    create_mode high_freq_mode
    create_clock -period 3.5 [get_port my_clock_port]

    create_mode low_freq_mode
    create_clock -period 7.5 [get_port my_clock_port]
    ```
  - If the two clocks enter from different ports and go through a clock MUX we have to use the `set_case_analysis` command
    - `set_case_analysis` sets specific conditions for signal values during the analysis. It fixes certain signals to specified logic values (0 or 1) to simulate specific operational modes.
    - In the diagram we need to set the CLK_SEL to 0 in the high-frequency mode and to 1 in the slow-frequency mode
    - ```
      create_mode high_freq_mode
      create_clock -period 3.5 [get_port my_clock_port]
      set_case_analysis 0 [get_port CLK_SEL]

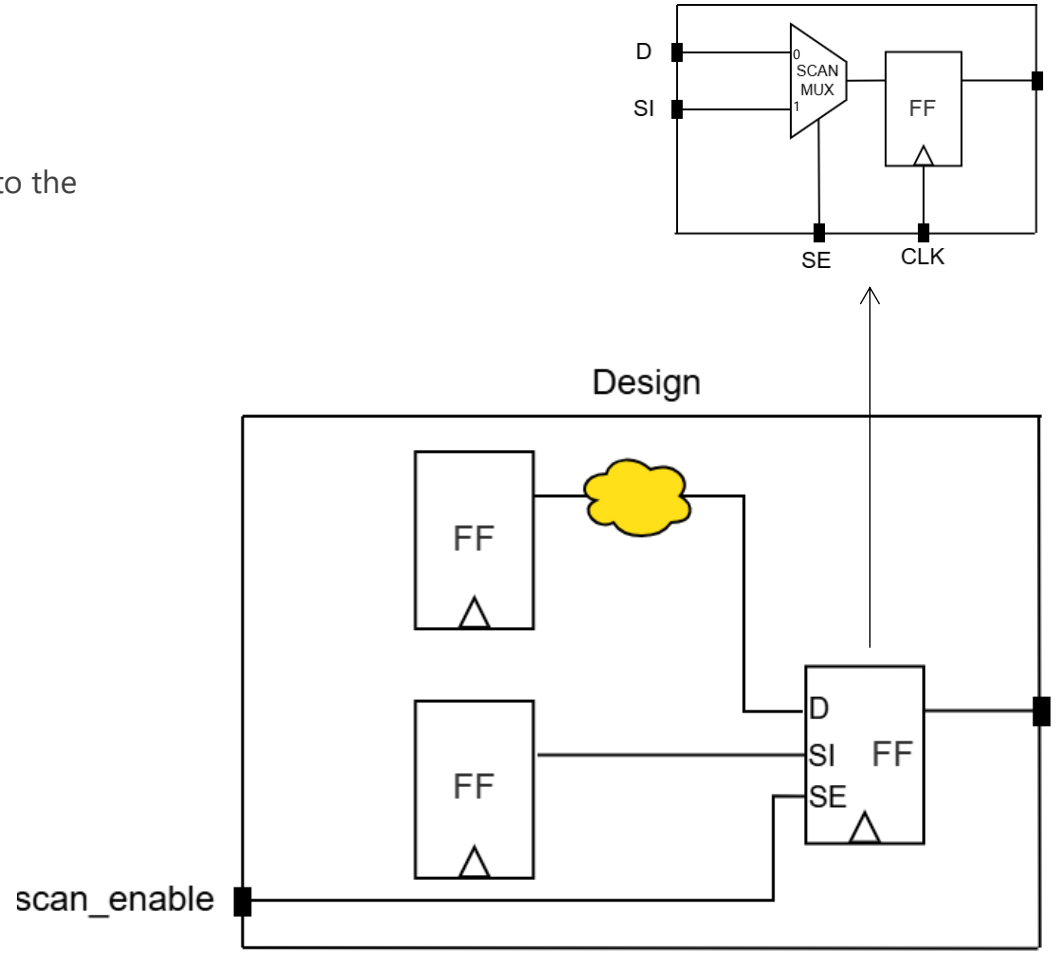
      create_mode low_freq_mode
      create_clock -period 7.5 [get_port my_clock_port]
      set_case_analysis 1 [get_port CLK_SEL]
      ```



# Modes – Scan Modes

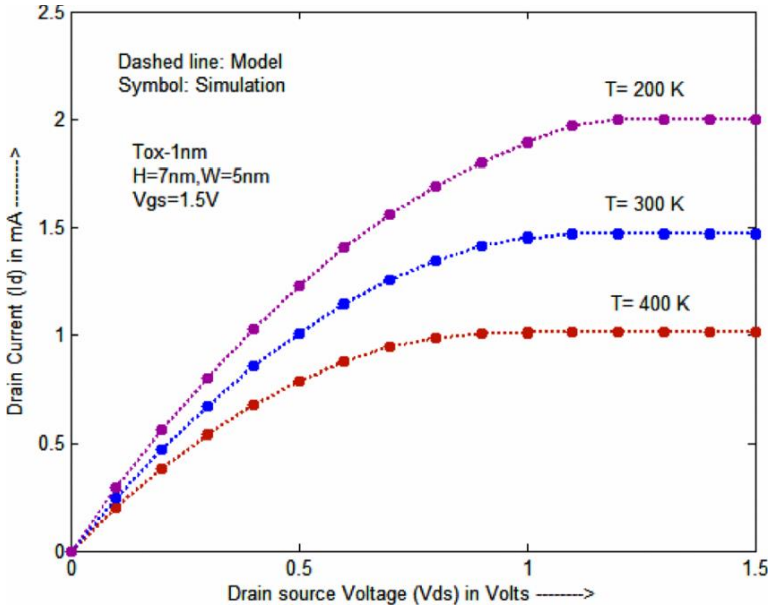
- Another usage for `set_case_analysis` is to enable or disable scan modes.
  - In functional modes we want the FF to receive input through the D pin
  - In scan mode we want the FF to receive input through SI (scan input) pin
  - This desired behavior can be selected by setting the scan enable port/pin to the correct value
  - 
```
create_mode functional_mode
create_clock -period 3.5 [get_port my_func_clk_port]
set_case_analysis 0 [get_port scan_enable]

create_mode scan_mode
create_clock -period 10 [get_port my_scan_clk_port]
set_case_analysis 1 [get_port scan_enable]
```



# PVT Corners

- PVT stands for process, voltage, and temperature variations and discusses the different fabrication, operating, and environmental conditions affecting the chip
- **Process:** Systematic and large variations such as doping, oxide thickness, etc can affect entire parts of a wafer resulting in all instances within the chip being faster or slower than average
- **Voltage:** Difference in the supply voltage provided to the entire chip. For example, a chip can have two operating voltages 3V for high performance mode and 2V for low power mode. The voltage affects the performance of the cells.
- **Temperature:** Ambient temperature where the chip will be operated. For example, the chip is being used in summer vs winter or in a cooled server room ( −40℃) vs a hot car engine (120℃). The temperature also affects the performance of the cells
- We run STA on all the different PVT corners to ensure the chip can operate correctly under all conditions.

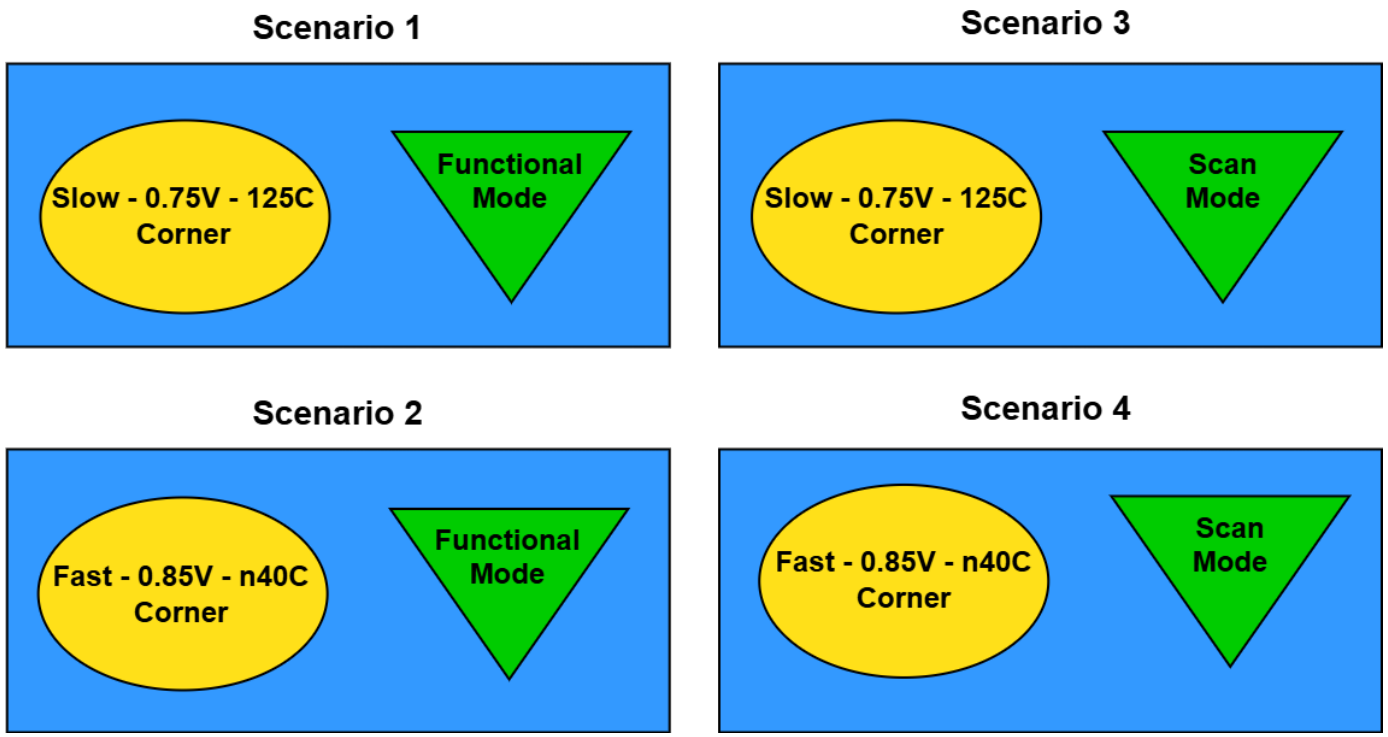



**MOSFET Current vs Voltage and Temp[1]**

[1]: Reference : **New analytical model for nanoscale tri-Gate SOI MOSFETs including quantum effects**

# Scenarios

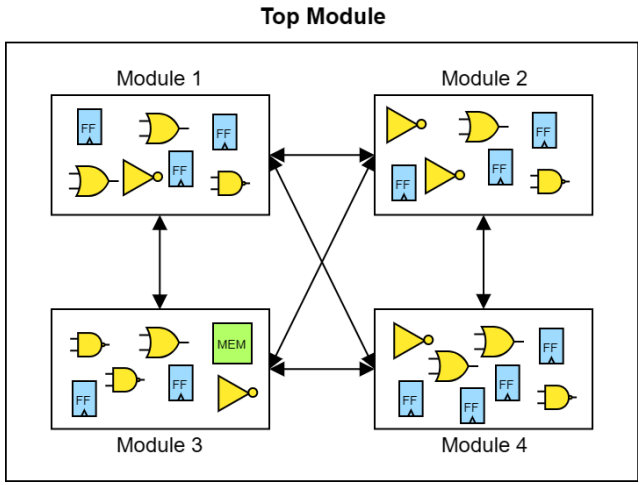
- An STA scenario is a combination of a corner and a mode.
- In general we run all the possible combinations. So, if there are 3 modes and 4 corners we run 12 scenarios.
- We can prune away some scenarios that won't happen to save time and resources. For example, if scan testing occurs in a controlled lab environment with low temperature, then high ambient temperature scenarios can be excluded for scan modes.



# Additional Topics

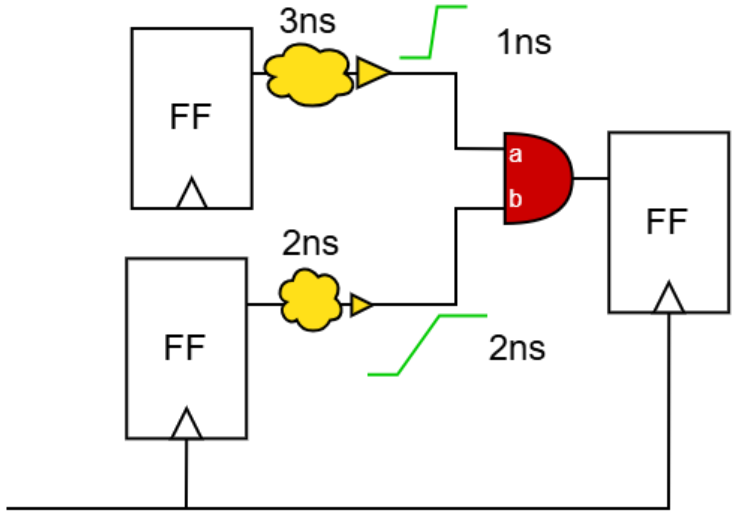
# Path Groups

- The design consists of several modules and blocks that communicate with each other.
- **It's common to categorize the timing path into groups based on the start and end points of the timing paths.**
  - For example, create a group for all timing paths going from **module_1** to **module_3**
    - `group_path -name FROM_MD1_TO_MD3 -from [get_pins -hier module_1/*CK] -to [get_pins -hier module_3/*D]`
- **Benefits of Path Grouping:**
  - Simplified Analysis: Easier and better analysis of the design, allowing identification of which module is causing more violations and needs redesign.
  - Team Collaboration: Enables multiple engineers to analyze the same design by assigning each engineer specific groups to analyze.
  - Giving Higher Priority For Optimizations: Timing paths within a group are optimized together by the tool. You can apply a higher weight/priority to a specific group so that the tool can put more focus on. The higher the weight the higher the effort applied to the group.
    - `group_path -name INOUT -weight 5 -from [all_inputs] -to [all_outputs]`



# Graph-Based Analysis (GBA) vs Path-Based Analysis (PBA)

- **Consider the example shown:** We have 2 timing paths going though the same AND gate and then a FF.
  - The upper path:
    - has a long combinational delay (3ns) but has a strong driver at the end.
    - The strong driver provides better transition time for the AND gate resulting in a delay of **1ns** within the AND.
    - That means a total delay of (4 ns) which doesn't violate setup.
  - The lower path
    - has a shorter combinational delay (2ns) but has a weak driver at the end.
    - The weak driver provides bad transition time for the AND gate resulting in a delay of **2ns** within the AND.
    - That means a total delay of (4 ns) which doesn't violate setup.
  - The problem is that STA tools, to reduce runtime, use a single transition time for both paths and it's the worst (The one resulting in **2ns** delay).
  - This means the upper path will, falsely, have a delay of (3ns + **2ns**) = 5ns which violates setup.
  - When the tools use a single worst transition time we call this graph-based analysis (GBA) and when it uses different transition times we call this path-based analysis (PBA).
  - GBA is used in most of the flow to reduce runtime. But during signoff, we run STA with PBA to remove the pessimism and avoid fixing any false violations.



# Thank You!